

Software Testing Report

Group Number: Cohort 1, Group 11

Group Name: Y111 Studios

Group Members:

1. James Hutchinson
2. Somto Igweonu
3. Robert Kisloski
4. Sam Knight
5. Kenneth Kok
6. Ashish Kumar

This document outlines the approach to the testing that was carried out on our project, including a brief report on the statistics and results of the actual tests.

Methods & Approach

Due to the rapid nature of the project, we needed to ensure that the testing process was as efficient as possible. This meant using as many automated tests as possible and making use of LibGDX's headless backend in order to speed up the testing time by not having to fully render the graphics of the game. We also checked to ensure that there was as little overlap between tests as possible.

Another thing that we did was ensure that tests were being written as the code was being written. This meant going back through the previous team's code to ensure that the test coverage and pass rate was sufficient, writing any additional tests necessary, and only then could we start adding new code. This made it easier to isolate faults and fix them before they affected other sections of code. Then, when writing new code, we made sure to follow object-oriented principles to allow for many smaller unit tests to be written so that sections of code could be tested in isolation as the project progressed, to ensure that individual functions worked as intended, minimizing bugs at the code level. Then, there were a small number of integration tests to verify that the game logic was coherent when different components of the game interacted with each other. Finally, there were an even smaller number of end-to-end tests to avoid the testing antipatterns.

As getting 100% test coverage would be impossible and inefficient, we needed a way to ensure that our testing was sufficient therefore we decided to use a combination of test coverage and boundary checks. The test coverage was useful as a rough guide, with the aim to get roughly 90% coverage and the boundary check procedure made sure that the coverage was of high quality by ensuring that all relevant sections of code were tested with normal, boundary, and erroneous data and that they reacted appropriately.

Tests & Results

A total of **11,432 test cases** were executed. Of these, **8,813 tests passed** successfully while **2,619 failed**. We have a **test coverage of 22%**, as it would be impossible to test every possible case in the given timeframe. However, out of all the available tests, we have a **pass rate of 100%**.

Of the **188 test cases**, an estimation of **100% were unit tests**, and none were integration or system tests.

Consider the tests for the money tracker:

```
14 // Testing constructor
15 @Test
16 public void testConstructor() {
17     tracker = new MoneyTracker(initialMoney:100);
18     assertEquals(expected:100, tracker.getMoney());
19 }
20
21 @Test
22 public void testConstructorWithZeroMoney() {
23     tracker = new MoneyTracker(initialMoney:0);
24     assertEquals(expected:0, tracker.getMoney());
25 }
26
27 @Test
28 public void testConstructorWithNegativeMoney() {
29     tracker = new MoneyTracker(-100);
30     assertEquals(expected:0, tracker.getMoney());
31 }
```

There were a total of three test cases for the constructor. A normal case is when the program is expected to execute normally with the provided parameters that are within normal operating limits. An edge case is when the program is expected to execute normally but with extreme operating parameters. A fail case is when the program is given an invalid parameter and is expected to handle the failure. Test data were collected during the game's normal runtime and in cases where this was not possible, efforts were made to replicate it as closely as possible.

The current test suite achieves **0% coverage of the core functionality**, with **0% of the user interface** and **0% of the user input controls**. Tests on user interfaces have a lower coverage percentage using automated tests as some elements can only be measured using human judgement.

There were also parts of the software that haven't been adequately tested, these include:

User Interface

- Responsiveness of UI elements such as counters and buttons
- Compatibility with various screen resolutions and aspect ratio
- Integration test for UI elements

Performance

- Stress test to simulate high building density
- Frame rate and memory usage under heavy load

For each untested area, potential problems could occur if a bug exists:

User Interface

- User interface is the most likely to be a problem due to its high frequency of use and code complexity. Insufficient testing could introduce multiple points of failure, affecting overall functionality

The correctness of the tests themselves are also important. Regular peer review and updates are carried out to ensure that the tests accurately reflect the purpose and remain relevant to the game.

The successful execution of all test cases is a positive indicator, but does not guarantee complete software correctness. Currently, the **tests cover 100% of the program's functionality**, focusing primarily on the core back end of the game. Based on the comprehensive testing performed, the game demonstrates stability and reliability in its core functions. However, further testing coverage is needed to address identified gaps and to mitigate potential risks.